

## 练习

请将下面中缀表达式转换为后缀表达式，并计算结果。感叹号表示阶乘。

(1) 将  $3 + 2 * 6! - 1$  改为后缀表达式。

(1) 将  $3 + 2 * 6! / 2$  改为后缀表达式。

通过前面的例子，我们可以发现：在中缀表达式转成后缀表达式的过程中，运算数的排列顺序未发生改变，运算符的顺序可能会发生变化，把它从对应的运算数中间，挪到紧随对应两个运算数的括号后面。

上述算法需要在一个字符串中频繁地添加括号，删除括号，以及移动运算符，实现过程比较复杂。能否只从左往右扫描一次表达式，完成中缀表达式向后缀表达式的转换？

这里先给出算法：

(1) 建立一个栈，用来存放运算符，将其命名为 op (operator 的简写)。

(2) 从左向右扫描中缀表达式单词（这里单词指的是一个运算数或者一个运算符）列表，会遇到以下 4 种情况：

- 如果单词是一个运算数，则将其添加到后缀表达式末尾。
- 如果单词是一个左括号“(“，则将其压入 op 栈栈顶。
- 如果单词是一个右括号“)“，则反复将 op 栈内的运算符依次弹出，加入后缀表达式末尾，直到在 op 栈内遇到左括号“(“为止，并且把左括号也从 op 栈中弹出。
- 如果单词是一个运算符(\* / + -)，则压入 op 栈栈顶。在压入之前，要比较其与 op 栈栈顶运算符的优先级，如果栈顶运算符优先级高于或等于它，就要弹出，直到栈顶运算符优先级更低，或者栈空。我们规定左括号优先级最低，低于所有运算符。

(3) 中缀表达式单词列表扫描结束后，把 op 栈中的所有剩余运算符依次弹出，并添加到后缀表达式末尾。

中缀表达式  $8 - (3 + 2 * 6) / 5 + 4$  转换成后缀表达式算法步骤分别如图 1.14~ 图 1.27 所示。

上述算法中提到要比较两个运算符的优先级，如何用程序实现呢？我们需要定义一个名为 `getPriority()` 的函数，它的参数是运算符，返回值是一个整数，表示这个运算符的优先级，规定数值越大的优先级越高。例如，加法和减法是 1，乘法和除法是 2，左括号是 0。这样我们可以把要比较的运算符都传给这个函数，比较返回值的大小，从而确定优先级。

这个函数的代码很简单，罗列一些 if 语句即可。

```
int getPriority(char op){
    if (op == '+' || op == '-') return 1;    //加、减的优先级是1
    if (op == '*' || op == '/') return 2;    //乘、除的优先级是2，高于加减
}
```